

Anthrop Management Limited — HRMS Build

Blueprint (v2)

Revised: 29 August 2026 — supersedes v1 **Change:** v1 was written without the earlier planning conversation. This version folds it in, keeps what was right in it, and says plainly where I disagree.

1. What changed from v1

The earlier discussion was planning a **full HR Management System** — employees, attendance, leave, onboarding, performance, training, payroll — with the ATS as one large module inside it. My v1 planned a **recruitment operations platform** with staff reporting bolted on.

Both are defensible. They are not the same product, they do not cost the same, and you cannot half-build both.

What I've kept from the earlier plan, because it was right:

- The layered architecture (foundation → HR core → modules → AI layer → reporting). Correct, and it is what v1 was describing in different words.
- **Build in stages, never all at once.** Fifteen modules at 10% each is the standard way these projects die.
- Payroll last. Nigerian statutory calculations — PAYE, pension, NSITF, NHIA — carry real financial and legal risk, and the rules change.
- **Do not merge OpenCATS with the AI Resume Analyzer.** That integration is the most expensive work in the whole plan and produces nothing a client would pay for.
- Puter.js is fine for a demo and wrong for real candidate data. Agreed, completely.
- The tiered AI architecture: extract with code, filter with rules, use AI only where interpretation is genuinely required. This is the single best idea in the earlier conversation.
- Vercel's Hobby plan forbids commercial use. Both analyses landed on this independently; it is correct and it rules Vercel out.

- Onboarding and offboarding as the joint between the ATS and the HR side. I missed this in v1 and it matters.
- The applicant portal, the careers page, and consent capture at the point of application. Also missed in v1.
- NDPA obligations, retention policy, and a DPIA assessment. Correct and non-optional for a business holding this data.

Where I disagree, in detail, is section 4.

2. The decision that changes everything

Before any code: **who is this system for?**

Scope A — Anthrop’s recruitment operations platform. You and your consultants run recruitment for Anthrop and other client companies. Clients log in to watch their own vacancies. Your own staff file daily and weekly reports and are measured on their work. The employees in the system are *yours*.

Scope B — a multi-tenant HR platform other companies use for their own staff.

Anthrop, Company B, Company C each log in and run their own employee database, attendance, leave, appraisals, and eventually payroll. Recruitment is one module among many. This is a software product, sold by subscription.

The earlier conversation was excited about Scope B, and it is genuinely the more commercially interesting direction. But be clear about what it means: attendance and leave are only valuable to a company managing its own staff daily. Building them for Anthrop’s own staff is a lot of work for a small return. Building them as a product means support, onboarding other companies, uptime commitments, and a sales process — a different business from HR consulting.

My recommendation: build Scope A, on Scope B’s foundations.

Concretely, that means every table carries a `tenant_id` from the very first migration, and the code never assumes there is only one organisation. That costs you almost nothing now and keeps the door to Scope B fully open. What you *don’t* do now is build attendance clocking, leave balances, or payroll for a use case you don’t yet have.

The earlier advice — “don’t hard-code it for Anthrop” — is right. But “design for multi-tenant” and “build fifteen modules” are two different instructions, and only the first one is free.

3. The shape of the system

Four layers, as the earlier plan had it:

Foundation — tenants, users, roles, permissions, audit log. **People core** — the record layer. **Modules** — ATS, staff reporting, onboarding, performance, and later attendance, leave, training, payroll. **AI layer** — sits across the modules, assists, never decides. **Reporting** — dashboards and exports over everything above.

One structural change I'd make to the earlier design

The earlier plan has "Employee Management" and "Recruitment/ATS" as separate modules with separate databases underneath. That is how most HR systems are drawn, and it creates a specific problem: when a candidate is hired, someone re-types their information into the employee module. Their recruitment history — how they were screened, how they interviewed, what the panel said — is severed from their employee record.

Better: **one people table**. A person can hold a *candidate* relationship to one or more jobs, and an *employee* relationship to one or more organisations. Hiring someone does not copy them anywhere; it adds a relationship.

What this buys you:

- Onboarding is a status change, not a data-entry task.
- An employee's original CV, screening score, and interview report stay attached to them for their whole employment. Two years later, when they're up for promotion, the panel's original assessment is one click away.
- If they leave and reapply in four years, the returning-applicant flag fires — because it's the same record.
- Rehiring a former employee, or hiring a former candidate for a different client, both work with no special handling.

This is one design decision made in week one that removes a whole category of duplication later.

4. Where I disagree with the earlier plan

4a. On model choice — right idea, wrong tier assignment

The earlier analysis ranked models by price and put the cheapest first: Gemini Flash-Lite,

then Mistral Small, then GPT-5 nano, with stronger models held back for “hard cases.” The arithmetic on token costs is sound, and the cost-control instinct is right.

But it treats “CV analysis” as one task. It is at least three, with very different requirements:

Task	What it needs	Right tier
Normalising messy text — is “People Operations” the same as “HR Management”?	Pattern recognition	Cheapest model. Volume is high, judgement is low.
Structured extraction from a CV into fields	Reliable formatting, no invention	Cheap model , with a schema and validation on the way out
Writing the assessment that goes to a client	Judgement, scepticism, holding a claim against its evidence	Strongest model available. This is the deliverable.

The third row is where the cost logic breaks down. Your screening skill exists because of a documented failure: a candidate ranked first at 89% finished last at interview, because a comma-separated skills list was scored as though it were evidence. The screen had *already noticed* his was the least checkable CV of the four and weighted that observation at 5%.

The fix for that failure is not a cheaper model. It is a model that can hold “he claims this” apart from “there is evidence for this” across a long document — and that is precisely the capability the cheap tier lacks. On a 200-CV role the difference between the cheapest model and the strongest one for the final assessment is a few dollars. A bad shortlist delivered to Anthrop costs considerably more than that.

So: tier by task, not by budget. Cheap models do the volume work — extraction, normalisation, deduplication support. The strong model writes assessments and interview analyses. Roughly 80% of your calls land on the cheap tier and 90% of your quality comes from the other 20%.

Two practical notes. First, the prices quoted in the earlier conversation move often — check them before committing. Second, benchmark on your own material: take 50 CVs from a role you’ve already completed, run each candidate model against them, and compare to the shortlist you actually produced and the interview outcomes you actually observed. That comparison is worth more than any pricing table.

4b. On rule-based pre-filtering — right, with a boundary

The earlier plan proposes: extract with code, filter with rules, then send only survivors to AI.

Correct, and it will cut your bill substantially.

The boundary that must be drawn: **rules may eliminate, rules may never rank.**

- Legitimate: "This role requires CIPM. No CIPM found in the document. Flag as not meeting an essential requirement." That is a hard, checkable fact.
- Illegitimate: "CV mentions Excel four times, so score the tools competency at 20%." That is keyword counting, and it is exactly what a stuffed CV is engineered to beat — and what your integrity scanner exists to catch.

Two further guards on the rules layer:

- A rule that eliminates must be reviewable. "Missing CIPM" should put a candidate in a *rejected-by-rule* bucket that a human can open, not delete them from the run. Certifications sit in the middle of a document, get abbreviated oddly, and sometimes appear only in a cover letter.
- **The integrity scan must run before the rules, on the raw file bytes.** Your existing skill is emphatic about this and it is right. A CV carrying 9,170 characters of white 2pt text will sail through a keyword filter — the hidden text is exactly the keywords the filter wants. Reading the file as text and applying rules to it, without scanning first, means the stuffed CVs are the ones that survive your filter.

4c. On OpenCATS

Both analyses reached the same conclusion by different routes. Confirming it: read the data model, don't install it, don't integrate it. Its own documentation says the CV parsing was lost and hasn't been replaced, which removes the one thing you'd most want from it.

4d. On Replit

The earlier plan recommended Replit as the development environment. That advice was written before Claude Code was in the picture. You already run Claude Code against your real estate platform on Render with GitHub — that workflow is better here: your code lives in a repository from the first commit, you review changes as diffs, and there's no migration from a prototype environment to a real one. Skip Replit.

4e. On Cloudflare D1 versus Supabase

The earlier plan floated Cloudflare Pages + Workers + D1 + R2, and noted Supabase as an easier alternative. Supabase is the better call here for one reason above all: **it gives you authentication and row-level security you don't have to write.**

With D1 you would build your own login handling, session management, password reset, and

permission enforcement. Every one of those is a well-understood way for a small platform to get breached, and permission checks scattered through application code are the specific failure that leaks one client’s candidates to another.

Supabase lets you write the rule once — *a client account can read only rows where tenant_id equals their own* — and the database enforces it. A bug in a screen cannot get around it. For a system holding several clients’ hiring data, that is the difference between an incident and a bug.

Cloudflare Pages for the frontend, though: agreed. Free, fast, and commercial use permitted.

4f. On CV retention versus returning-applicant detection

This is a genuine conflict between the two plans, and neither spotted it.

The earlier plan says: don’t keep CVs forever, set a retention period, delete automatically. That is correct under the NDPA and it is good practice.

Your stated top priority says: recognise anyone we have processed or interviewed before, so we know whether to invite them again. That requires memory that outlives any short retention period. A twelve-month deletion rule means someone who applied thirteen months ago is a stranger.

The resolution: split the record.

Kept	For how long	Contains
The CV file and full parsed profile	The retention period (e.g. 24 months from last activity)	Everything — employment history, addresses, salary expectations, the document itself
The processing history record	Longer, e.g. 7 years, or until the person asks for erasure	Name, contact identifiers, which role, when, stage reached, outcome, the recorded reason, and the CV fingerprint

The second record is small, minimally intrusive, and is all the returning-applicant flag actually needs. When the CV is deleted, the flag still fires — it just says “processed for Operations Associate in Nov 2025, screened out at CV stage; CV no longer held” instead of showing you the document.

This position is defensible to a regulator (you are keeping the minimum necessary, for a stated purpose, for a stated period), defensible to a candidate, and it gives you the feature.

Have Anthrop's people confirm the actual periods — I'm describing the structure, not setting the policy.

5. The modules

Grouped by the layer they sit in. **Scope A** modules are in the build now. **Scope B** modules are designed for but not built.

Foundation

M0 — Tenants, users, roles, audit (*Scope A*) Tenant records, user accounts, five roles (Owner, Consultant, Interviewer, Staff, Client), permission enforcement in the database via row-level security, and an audit log written by the database rather than by screens. Every table that holds tenant data carries `tenant_id` from migration one.

People core

M1 — People, clients, jobs (*Scope A*) The single `people` table described in section 3, plus `client_organisations` and `jobs`. Job descriptions are broken into a scoreable criteria list at creation time, each line marked essential or desirable with a weight — because that list is what screening scores against, and generating it later is how screening drifts back into generic seniority ranking.

M2 — Applications, CV intake, applicant portal (*Scope A*) Bulk CV upload from a folder, plus a public careers page where candidates apply directly. The portal is where consent is captured — a plain-language notice of what you'll do with their CV, how long you'll hold it, and that AI assists the screening, with the acceptance timestamped and stored against the application. Consent has to be captured at collection; it cannot be added later.

A CV upload never creates a person directly. It creates a *match attempt* which either links to an existing person or creates a new one, and tells you which it did. Names are matched by folding case, stripping punctuation, and comparing sorted token sets — never on surname alone. Nigerian phone numbers are normalised so `08031234567` and `+2348031234567` are one number.

M2b — Returning applicant detection (*Scope A — your priority item*)

Four independent checks. One firing raises a flag; two or more makes it a strong match.

1. **Contact match** — same email or same normalised phone. Near-conclusive.
2. **Name match** — normalised token set matches. Weak alone; presented as "possible," needs confirmation.

3. **File hash** — SHA-256 of the uploaded file. Catches the identical file resent under a new name.
4. **Content fingerprint** — similarity across stripped text, plus a signature from the employment history (employer names with start and end dates). This is the one that catches the re-saved, lightly edited CV a year later, which is the common case.

What you see, before reading anything else:

Previously processed — 2 prior applications • *Branding & Communications Manager* (Mar 2026) — reached Interview. Outcome: **Not recommended**. Reason: "Could not evidence campaign ownership claimed on CV." • *Operations Associate* (Nov 2025) — screened out at CV stage, 41% match. CV no longer held (retention). On file: **Review before inviting**.

Rules:

- **A flag never auto-rejects**. It surfaces history and stops. Someone rejected eighteen months ago for lacking a skill may have it now — and your own screening rules already treat evidence as time-sensitive.
- The **do-not-invite list** is a deliberate action requiring a reason, the person deciding, and an expiry (default 24 months, renewable). Reasons must be job-related and written in language you'd be comfortable showing the candidate, because under the NDPA they can ask.
- Cross-client history is visible to your consultants and **never to another client**. That history is your asset.

M2c — Historical import from Google Drive (*Scope A — do this early*) Walks your existing role folders, parses every CV, deduplicates within and across folders, creates historical people and application records with the outcomes you recorded. Read-only against Drive — it never moves or modifies source files. **Paginate**: a folder of 26 CVs returns 10 on the first page, and stopping there silently imports a third of your history.

Without this, returning-applicant detection knows nothing on launch day.

Recruitment modules

M3 — Screening and shortlisting (*Scope A*) Your `candidate-shortlist-report` skill as a feature. Integrity scan on raw bytes → rule-based elimination on essentials → cheap-model extraction → strong-model assessment on survivors → ranked table with evidence per score → falsification probes → overqualified bucket → misfiled applications listed separately → human approves the shortlist → .docx export.

Every score traces to a line in a specific CV. Weight what is expensive to fake; discount what is free to write.

M4 — Interview management (*Scope A*) Scheduling, panel assignment, the pre-interview questionnaire as a link the candidate fills in, and **the panel scoring sheet as a form each panel member completes on their phone.**

That last item is the biggest single quality win available. Scores enter the system as data, from the person who gave them, at the time they gave them. Every downstream error involving scores — mismatched rows, transcription slips, estimated numbers — comes from scores living in a spreadsheet that has to be read back later. Cohort rankings are computed once across the full set and passed down.

M5 — Post-interview analysis and reports (*Scope A*) Your `interview-summary-report` pipeline, with its gates preserved as approval screens.

Stage	Human gate
Upload raw Zoom/VTT	—
Clean transcript	Yes
Extract (CV + questionnaire + panel scores + transcript)	Yes
Analyse against the JD	—
Build .docx	—
Verify automatically	Yes
Publish to Drive	—

Unbreakable rules: cleaning removes noise and never adds substance, so a vague answer stays vague. Scores are read, never derived — a missing panel score stops the report rather than being estimated. Contradictions are surfaced, not silently resolved. Layout lives in `build_report.js` ; nobody hand-edits a report.

M6 — Offer, onboarding, offboarding (*Scope A — the joint*) Offer issued → accepted → the person's employee relationship activates → onboarding checklist, documents collected, department and manager assigned. Their CV, screening score, and interview report stay attached because it is the same record.

Offboarding runs the same way in reverse: resignation or termination → exit interview → property returned → **access disabled** → archived. The access-disabled step is a security control, not an admin task; make it a single button that actually revokes the login.

M7 — HR documents (*Scope A, small*) Contracts, offer letters, ID documents, certificates, policies, appraisals. Permissions: HR sees all, a manager sees their team's, an employee sees their own. Files stored with random names, no public URLs, authenticated download only, access logged.

Staff modules

M8 — Daily and weekly reports (*Scope A — your requirement*) The daily form must be completable in under two minutes on a phone. Three fields, not fifteen: what you worked on, what's blocking you, what's next. A long form is abandoned by week three; a short one becomes a habit. Weekly reports pre-fill from that week's dailies, so the weekly is edit-and-confirm rather than a blank page. Reminders at end of day, manager summary Monday morning. Late is recorded as late; missing is never auto-filled.

M9 — Performance tracking (*Scope A — your requirement*) Two halves. Automatic measures drawn from work the system already records: roles handled, candidates screened, time from job opening to shortlist delivered, shortlist-to-interview conversion, report turnaround, reports returned for correction, report submission punctuality. And manager judgement: goals set jointly before the period, self-assessment, review, sign-off.

Two rules:

- **Every staff member sees their own numbers, always.** Measurement people can't see is surveillance; measurement they can see is feedback — and hidden metrics get disputed the moment they're used in a review.
- Automatic numbers never produce a rating alone. A consultant assigned three hard roles will lose on raw conversion to one assigned three easy ones.

M10 — Client portal (*Scope A*) Clients see their own tenant only, enforced by the database. Approved and published items only — no drafts, no raw transcripts, no screening internals, no cross-client history. Candidate contact details hidden until the shortlist is formally accepted. Client decisions recorded with timestamps.

M11 — Search, dashboards, notifications (*Scope A*) Full-text search across people and CVs ("who have we seen with FMCG brand management experience?"), operations dashboard, transactional email.

M12 — Hardening, privacy, launch (*Scope A — before real client data*) Nightly backup to somewhere that is not the database host. Rate limiting on login and public forms. Two-factor for Owner and Consultant. Upload controls: file-type whitelist, size limits, malware scanning, randomised filenames. Retention and automatic deletion running on schedule. Privacy notice, candidate data-request route. Error monitoring, health checks, rollback, and a written runbook. And a decision, with Anthrop's legal input, on whether a DPIA is required

— AI-assisted recruitment is exactly the kind of processing the NDPC guidance points at.

Scope B — designed for, not built now

Attendance and time — clock in/out, schedules, lateness, overtime. **Leave** — requests, approvals, balances, calendar, categories. **Training** — catalogue, requests, attendance, certificates, skills gaps. Genuinely relevant given Anthrop's training business. **Employee self-service** — the employee-facing side of all of the above. **Payroll** — last, and only with proper advice. PAYE, pension, NSITF, NHIA. Financial and regulatory risk, and the rules change.

Each of these is 2–4 weeks of work. Build them when a paying user asks for them, not before.

6. The AI pipeline

```
CV uploaded
  ↓ validate file type and size
  ↓ INTEGRITY SCAN on raw bytes           ← no AI, and this must come first
  ↓ extract text (PDF/DOCX library)       ← no AI
  ↓ OCR only if the text layer is empty    ← not a language model
  ↓ clean text                            ← no AI
  ↓ pattern extraction: email, phone,
    dates, known certifications           ← no AI
  ↓ CHEAP MODEL: structure the rest into
    schema-validated fields               ← high volume, low judgement
  ↓ RULES: essential requirements met?
    → not met = rejected-by-rule bucket (reviewable, not deleted)
  ↓ STRONG MODEL: assessment against the
    job criteria, with evidence and gaps ← the deliverable
  ↓ store result, keyed to person + job
  ↓ RECRUITER REVIEWS AND DECIDES
```

Three cost rules from the earlier plan, all worth keeping:

- **Analyse once, store the result.** Never re-read a CV because someone opened a screen. Re-run only when the job's criteria change.
- **Store the person separately from the job.** Comparing person 123 against job 456 shouldn't require re-reading the document.
- **Never send a whole CV where a structured profile will do.** The profile is a fraction of the tokens.

And the safeguards, which are as much commercial protection as ethics: the AI never says "hire this person," scores are always broken out with evidence rather than given as a single number, job-related criteria only — never gender, ethnicity, religion, photograph, marital status, or name-based assumptions — every output stored with a timestamp and the model version, and a recruiter can override anything.

7. Build order

#	Module	Why here
1	M0 Foundation	Tenant scoping and audit are cheap now, a rebuild later
2	M1 People, clients, jobs	The record layer everything attaches to
3	M2 Applications + portal + consent	Consent must exist before real CVs arrive
4	M2b Returning-applicant detection	Built while the matching logic is fresh
5	M2c Historical import	Early. Detection needs history, and the import will expose data problems you want to find in month two, not month eight
6	M3 Screening	Your highest-volume manual task. First real hours saved
7	M8 Daily/weekly reports	Deliberately out of sequence — small, independent, gets your team using the platform daily while the recruitment side is still being built
8	M4 Interview management	Turns scoring sheets into data, which M5 needs
9	M5 Analysis and reports	Most complex. Build once its inputs arrive clean
10	M6 Offer and onboarding	Closes the loop from candidate to employee
	M7 HR	

11	documents	Small, and onboarding needs somewhere to put files
12	M9 Performance	Needs months of activity to mean anything
13	M10 Client portal	Worth showing clients once the pipeline behind it is real
14	M11 Search and dashboards	Best built across a complete system
15	M12 Hardening and launch	Before real client data. Not after

Treat M0 → M3 as one push. At the end you have a working ATS with jobs, candidates, returning-applicant flags, and screening — the version you could run the next Anthrop role on.

8. Timeline

A working day is roughly 4–5 focused hours, with Claude Code doing the typing and you deciding, reviewing, and testing.

Scope A

Module	Working days	At 5 days/wk	At 2 days/wk
M0 Foundation	5–7	Wk 1–2	Wk 1–4
M1 People, clients, jobs	5–6	Wk 2–3	Wk 4–7
M2 Applications + portal	8–10	Wk 3–5	Wk 7–12
M2b Returning-applicant	4–5	Wk 5	Wk 12–15
M2c Historical import	3–4	Wk 5–6	Wk 15–17
— first usable ATS —	25–32	~Wk 7	~Wk 17
M3 Screening	8–10	Wk 7–8	Wk 17–22
M8 Daily/weekly reports	4–5	Wk 9	Wk 22–25

M4 Interview management	6–8	Wk 10–11	Wk 25–29
M5 Analysis and reports	8–10	Wk 11–13	Wk 29–34
M6 Offer and onboarding	4–5	Wk 13	Wk 34–36
M7 HR documents	3–4	Wk 14	Wk 36–38
M9 Performance	5–6	Wk 15	Wk 38–41
M10 Client portal	4–5	Wk 15–16	Wk 41–43
M11 Search and dashboards	4–5	Wk 16–17	Wk 43–46
M12 Hardening and launch	5–6	Wk 17–18	Wk 46–49
Scope A total	77–98	~18 weeks	~11–12 months

Scope B, if you go there later

Attendance and time 8–10 days · Leave 6–8 · Training 6–8 · Employee self-service 8–10 · Payroll 15–20 plus professional advice. Roughly **43–56 additional working days**, or another 9–12 weeks full-time.

Three honest notes. These assume Claude Code; without it, roughly double. They exclude what always overruns — your real data, which is badly scanned and inconsistently named, so add 20% across M2, M2c, and M3. And testing each module against a real completed role is included and not optional: a module that has only seen invented data fails on the first live job.

9. Hosting

Layer	Service	Free tier	Why
Frontend	Cloudflare Pages (or Netlify)	Free, HTTPS, commercial use allowed	Vercel Hobby forbids commercial use
Backend API	Render web service	512 MB, 750 hrs/month, no card	You already use it; runs a persistent Node process
Database, auth, files	Supabase	500 MB DB, 50k monthly users, 1 GB	Auth and row-level security you don't have to write

		files	
Scheduled jobs, backups	GitHub Actions	2,000 min/month private	Render's free tier has no cron or workers
AI	Model APIs	Pay per use	The one unavoidable cost

Four limits to design around:

1. **Vercel Hobby forbids commercial use.** Ruled out.
2. **Render's free database is deleted after 30 days** — not paused, deleted, after a 14-day grace period. The real database never goes there.
3. **Supabase pauses free projects after 7 days without database activity.** Data survives; the app goes offline until resumed. Fix with a GitHub Actions ping every couple of days — ten minutes of setup.
4. **Render free services sleep after 15 minutes and take 30–60 seconds to wake.** Fine while you're the only user; not fine when a client logs in. Budget **\$7/month** for Render Starter the week before clients get access.

Running cost: \$0 while building and for internal use, plus AI usage. About \$7/month once clients log in. Supabase Pro at \$25/month becomes worth it when you cross 500 MB or want automatic daily backups — and the earlier analysis was right that free-tier backups are the point where “~~no~~ production” stops being honest. Realistically that's a few hundred candidates with stored CVs.

Free-tier plan and upgrade triggers

The intention is to build the entire system on free tiers and pay only when there is a reason. That works, with one thing to expect rather than be surprised by.

The binding limit is file storage, not the database. Supabase free allows 1 GB of files. A CV runs 200 KB to 1 MB, so that is roughly 1,000–4,000 CVs. The historical import (M2c) may consume most of it in one go, and one high-volume role could finish it. The 500 MB database allowance is comparatively generous — extracted text takes almost no space.

When you reach it, two options: pay the \$25/month, or keep CV *files* in Google Drive and store only the extracted text and a pointer in Supabase. The second stays free longer and fits the existing Drive workflow, at the cost of more work around access control and audit.

Three points where paying stops being optional:

--	--	--

Trigger	What to pay for	Why
Real candidate CVs enter the system	Supabase Pro, \$25/mo	Free has no backups, and you are the data controller for hundreds of people's personal information
File storage passes ~800 MB	Supabase Pro, or move files to Drive	Don't wait for the wall
Clients are given logins	Render Starter, \$7/mo, plus the domain switch	Cold starts are unacceptable to an external user

Everything before those three — all of Module 1 and most of Module 2 — runs genuinely free.

Regardless of tier: run the nightly export from day one, and **restore from it once into a scratch Supabase project**. A backup nobody has restored is a file, not a backup. Find that out in month two rather than on the day it matters.

Keep Supabase Storage as the working store and continue archiving finished roles to Google Drive. Drive stays your long-term archive and client handover; it stops being the working system.

10. Building it with Claude Code

```
hrms/
├─ CLAUDE.md          # standing brief, read every session
├─ docs/
│   └─ blueprint.md   # this document
│   └─ modules/       # one spec per module
│       └─ decisions.md # every choice, and why
├─ frontend/
├─ backend/
├─ database/migrations/ # numbered, never edited once run
└─ tests/
```

CLAUDE.md is the highest-value file in the repository. It holds the stack, the layout, the conventions, and the rules that must never break:

- Scores are read, never derived. A missing value is “Not stated” plus a flag — never a plausible guess.

- Flags never auto-reject.
- Every table carries `tenant_id`. Row-level security on every one of them.
- Audit log on every write.
- Rules eliminate; rules never rank.
- The integrity scan runs on raw bytes, before any text is read.

The loop, once per module

1. Write the spec first, including what “finished” means as things you will click and check.
2. Ask for the database changes **alone**, and review the schema before any screen exists. Schema mistakes are the expensive ones.
3. Backend, then frontend, in separate sessions.
4. Tests on the rules, not the buttons. “A client user cannot read another tenant’s job” is worth a permanent test.
5. Test against a real completed role before moving on.
6. One branch per module, deploy at the end of each. Fifteen small deployments that each worked are recoverable; one big one that didn’t is not.

A kickoff prompt

Read `CLAUDE.md` and `docs/modules/00-foundation.md`. Build only the database schema for Module 0 — tenants, users, roles, audit log — as a numbered migration, with row-level security policies on every table holding tenant data. No screens, no endpoints yet. Then show me the schema and explain in plain language what each policy allows and blocks.

Four guardrails

- Never let generated code invent a score, a date, or a name.
 - Never let it edit a migration that has already run — new change, new numbered file.
 - Review every place data crosses a tenant boundary yourself. That’s the bug category that costs a client rather than an afternoon.
 - Keep report building deterministic. Layout comes from `build_report.js`, not from the model improvising per candidate.
-

11. Decisions needed before Module 0

1. **Scope A or Scope B?** My recommendation: A, on B's foundations. This is the one that changes the timeline by months.
2. **Retention periods** — for CV files, and for the processing-history record. Anthrop should set these with legal input; I've proposed the structure, not the numbers.
3. **Which completed role is the test case throughout?** Branding & Communications Manager looks right — it has CVs, questionnaires, transcripts, scoring sheets, and finished reports, so it exercises every module.
4. **Do you run the model benchmark first?** Fifty CVs from a completed role, three or four models, compared against the shortlist you produced and the interview outcomes you observed. Two or three days, and it settles the model question with evidence instead of a price list.
5. **Clients get logins at launch, or later?** Later is cheaper and safer; M10 is deliberately near the end.

This document is the plan of record. When a decision changes, change it here first, then in `docs/decisions.md`, then in the code.